



SYSTÈME DE RÉSERVATION (BOOKING)

Rapport de Développement Technique Complet

Réalisé par

YAHYA EL OMARI

Encadré par

MR. ABDELMAJID BENDRIF & MR. HAMZA BAHLAOUANE

github.com/yahyaelomarivoid/booking_drupal

Mars 2026

Table des matières

Introduction	2
Chapitre 1 : Setup and Finalize Module MVP	3
Chapitre 2 : Amélioration du Design UI	10
Chapitre 3 : Filtrage Dynamique	13
Chapitre 4 : Filtres, Tris et Pagination	15
Chapitre 5 : Refactorisation et Export CSV	17
Chapitre 6 : Optimisation Mémoire et Batching	18
Chapitre 7 : Symfony Mailer et Notifications	19
Chapitre 8 : Validation des Transitions	20
Chapitre 9 : Profiling Conseiller	21
Chapitre 10 : Gestion des Créneaux Horaires	22
Chapitre 11 : Interface Interactif (FullCalendar)	24
Chapitre 12 : Système de Soft Delete	25
Chapitre 13 : Système de Permissions	26

INTRODUCTION

Ce document retrace le développement complet du module Drupal booking. L'architecture du rapport est construite chronologiquement, chapitre par chapitre, selon les *commits* Git effectués sur le projet. Pour chaque étape clé, le code source principal ainsi que le résultat visuel sont exposés afin de bien comprendre l'évolution du module, allant du MVP (Minimum Viable Product) initial jusqu'à l'amélioration graphique de l'interface utilisateur.

CHAPITRE 1 : SETUP AND FINALIZE MODULE MVP

Dans cette phase initiale (Commit 1), nous avons créé les fondations du module. Le système gère la création dynamique de deux entités personnalisées : **BookingEntity** pour le stockage des requêtes de rendez-vous, et **AgencyEntity** pour structurer les lieux géographiques. La logique de réservation a été répartie à travers un service dédié (**BookingService**) et un formulaire multi-étapes.

1.1 Entité Principale (BookingEntity.php)

```
1 <?php
2 namespace Drupal\booking\Entity;
3 use Drupal\Core\Entity\ContentEntityBase;
4 use Drupal\Core\Entity\EntityTypeInterface;
5 use Drupal\Core\Field\BaseFieldDefinition;
6
7 /** @ContentEntityType(...) */
8 class BookingEntity extends ContentEntityBase {
9     public static function baseFieldDefinitions(EntityTypeInterface $entity_type): array {
10         $fields = parent::baseFieldDefinitions($entity_type);
11         $fields['reference'] = BaseFieldDefinition::create('string')->setLabel(t('Reference'))->setRequired(TRUE)->setReadOnly(TRUE);
12         $fields['booking_customer_name'] = BaseFieldDefinition::create('string')->setLabel(t('Customer name'))->setRequired(TRUE);
13         $fields['booking_customer_email'] = BaseFieldDefinition::create('email')->setLabel(t('Customer email'))->setRequired(TRUE);
14         $fields['booking_customer_phone'] = BaseFieldDefinition::create('string')->setLabel(t('Phone number'))->setRequired(TRUE);
15         $fields['booking_date'] = BaseFieldDefinition::create('datetime')->setLabel(t('Appointment date and time'))->setRequired(TRUE);
16         $fields['booking_agency'] = BaseFieldDefinition::create('entity_reference')->setSetting('target_type', 'agency')->setLabel(t('Agency'))->setRequired(TRUE);
17         $fields['booking_adviser'] = BaseFieldDefinition::create('entity_reference')->setSetting('target_type', 'user')->setLabel(t('Adviser'))->setRequired(TRUE);
18         $fields['booking_type'] = BaseFieldDefinition::create('entity_reference')->setSetting('target_type', 'taxonomy_term')->setLabel(t('Type'))->setRequired(TRUE);
19         $fields['booking_status'] = BaseFieldDefinition::create('list_string')->setLabel(t('Status'))->setSetting('allowed_values', ['pending' => 'Pending', 'confirmed' => 'Confirmed', 'cancelled' => 'Cancelled']);
20         return $fields;
21     }
22 }
23 }
```

1.2 Trait du Formulaire (BookingFormTrait.php)

Les étapes du formulaire ont été factorisées dans un 'Trait' PHP pour préserver la clarté du code et permettre son utilisation entre plusieurs classes.

```
1 <?php
2 namespace Drupal\booking\Form;
3 use Drupal\Core\Form\FormStateInterface;
4
5 trait BookingFormTrait {
6     public function buildAgencyOptionsStep(array &$form, FormStateInterface $form_state) {
7         $options = $this->bookingService->getAgencyOptions();
8         $form['agency_options'] = [
9             '#type' => 'radios',
10            '#title' => $this->t('Select an agency'),
11            '#options' => $options,
12            '#required' => TRUE,
13            '#ajax' => [ 'callback' => '::ajaxCallback', 'wrapper' => 'booking-form-wrapper' ],
14        ];
15    }
16    // Les methodes similaires (buildAdviserStep, buildServiceOptionsStep, etc.)
17    // sont appelees recursivement ici pures.
18 }
```

1.3 Assemblage Multi-Étapes (BookingSubmitForm.php)

```
1 <?php
2 namespace Drupal\booking\Form;
3 use Drupal\Core\Form\FormBase;
4 use Drupal\Core\Form\FormStateInterface;
5
6 class BookingSubmitForm extends FormBase {
7     use BookingFormTrait;
8
9     public function buildForm(array $form, FormStateInterface $form_state): array {
10         $step = $form_state->get('step') ? 1;
11         $form['#prefix'] = '<div id="booking-form-wrapper">';
12         $form['#suffix'] = '</div>';
13
14         switch ($step) {
15             case 1: $this->buildAgencyOptionsStep($form, $form_state); break;
16             case 2: $this->buildAdviserStep($form, $form_state); break;
17         }
18     }
19 }
```

```

17 case 3: $this->buildServiceOptionsStep($form, $form_state); break;
18 case 4: $this->buildDateTimeStep($form, $form_state); break;
19 case 5: $this->buildPersonalInformationStep($form, $form_state); break;
20 case 6: $this->buildConfirmationStep($form, $form_state); break;
21 }
22
23 $form['actions'] = ['#type' => 'actions'];
24 if ($step > 1 && $step < 6) { $form['actions']['back'] = ['#type' => 'submit', '#value' => $this->t('Back'), '#ajax' => ['callback'
=> '::ajaxCallback', 'wrapper' => 'booking-form-wrapper']]; }
25 if ($step < 6) { $form['actions']['next'] = ['#type' => 'submit', '#value' => $this->t('Next'), '#ajax' => ['callback' => '::
ajaxCallback', 'wrapper' => 'booking-form-wrapper']]; }
26 else { $form['actions']['submit'] = ['#type' => 'submit', '#value' => $this->t('Finish')]; }
27
28 return $form;
29 }
30 }

```

1.4 Mécanisme AJAX du Formulaire

Pour garantir une expérience utilisateur fluide, le passage d'une étape à l'autre ne nécessite aucun rechargement complet de la page. Cela est rendu possible grâce à l'utilisation du système AJAX natif de l'API Form de Drupal. Chaque bouton d'action déclare une propriété **#ajax** pointant vers un callback. Lorsqu'un bouton est cliqué, la soumission partielle déclenche le backend, incrémente l'étape courante, reconstruit le formulaire, et le callback renvoie le nouveau code HTML qui vient remplacer instantanément le conteneur `<div id="booking-form-wrapper">`.

```

1 public function ajaxCallback(array &$form, FormStateInterface $form_state) {
2   return $form; // Drupal replace automatiquement le "wrapper" par ce `$form` recalculé
3 }

```

1.5 Interface d'Administration (CRUD)

Le module implémente une interface complète pour la gestion des réservations côté administrateur, utilisant le composant **EntityListBuilder** de Drupal pour facilement lister, trier, et filtrer les données.

```

1 <?php
2 namespace Drupal\booking;
3 use Drupal\Core\Entity\EntityListBuilder;
4
5 class BookingListBuilder extends EntityListBuilder {
6   public function buildHeader(): array {
7     $header['reference'] = $this->t('Ref');
8     $header['booking_customer_name'] = $this->t('Customer');
9     $header['booking_date'] = $this->t('Date');
10    $header['booking_status'] = $this->t('Status');
11    return $header + parent::buildHeader();
12  }
13  // ... buildRow et logiques d'actions
14 }

```

1.6 Interface Client et Sécurité

Une vue dédiée (**BookingEditForm**) permet aux clients de modifier ou de consulter l'état de leur réservation existante sans créer de compte. Note de sécurité importante : Actuellement, dans ce MVP (Commit 1), l'accès à la modification se fait en fournissant uniquement la référence textuelle de la réservation. Il n'y a pas encore de sécurité robuste activée. Toutefois, l'architecture prévoit une évolution : prochainement, lorsqu'un client entrera sa référence en ligne, un code secret temporaire lui sera généré et envoyé par e-mail. Ce n'est qu'après avoir validé ce code (2FA) qu'il obtiendra le droit de modifier son rendez-vous.

Prendre un rendez-vous



Select an agency *

☐ Agadir Central Office

☐ Talborjt Branch

☐ Anza Tech Hub

Next »

FIGURE 1 – Step 1 : Select Agency (MVP)

Select an adviser *

☐ admin

☐ dev

☐ exportDev

☐ john_adviser

☒ jane_adviser

« Back

Next »

FIGURE 2 – Step 2 : Select Adviser (MVP)

Select a service *

☐

Follow-up Meeting

☐

General Consultation

☐

Initial Interview

☐

Technical Support

« Back

Next »

FIGURE 3 – Step 3 : Select Service (MVP)

Select a date and time *

21/03/2026 

02:01:00 

« Back

Next »

FIGURE 4 – Step 4 : Select Calendar Date (MVP)

Personal Information

Name *

Email *

Phone *

« Back

Next »

FIGURE 5 – Step 5 : Personal Info (MVP)

Confirmation

Please review your booking details below:

- **Agency:** Agadir Central Office
- **Adviser:** jane_adviser
- **Service:** General Consultation
- **Date:** 2026-03-21 02:01:00
- **Name:** yahya2
- **Email:** y.elomari@void.fr
- **Phone:** 0708521209

Finish

FIGURE 6 – Step 6 : Preview and Confirm (MVP)

Home > Administration > Structure

Booking List ☆

Reference	Customer	Date	Status	Agency	Operations
REF-20260317-3273	Ahmed Idrissi	2026-03-24T09:00:00	En attente	Agadir Central Office	View ▾
REF-20260317-3C75	Omar El Amrani	2026-03-27T13:00:00	En attente	Agadir Central Office	View ▾
REF-20260317-4565	Ahmed Idrissi	2026-03-25T16:00:00	En attente	Agadir Central Office	View ▾
REF-20260317-4E3F	Sanaa Mansouri	2026-04-04T10:00:00	En attente	Agadir Central Office	View ▾
REF-20260317-55C0	Ahmed Idrissi	2026-04-04T10:00:00	En attente	Agadir Central Office	View ▾

FIGURE 7 – Admin : Booking List Builder

Home > Administration > Structure > Booking List > REF-20260318-AE1B

Update Booking Status ☆

Booking details

- Reference: **REF-20260318-AE1B**
- Customer: **yahya2**
- Date: **2026-03-21T02:01:00**

Status *

Confirmé ▾

Save

FIGURE 8 – Admin : Update Status

Gérer mon rendez-vous



Enter your booking reference *

REF-YYYYMMDD-XXXX

Find my booking

FIGURE 9 – Client : Manage Booking Overview

Service *

Follow-up Meeting



Appointment date *

21/03/2026



02:01:00



Your name *

yahya2

Email *

y.elomari@void.fr

Phone *

0708521209

Save changes

Cancel my booking

FIGURE 10 – Client : Editing their Booking

CHAPITRE 2 : AMÉLIORATION DU DESIGN UI (CARDS)

Dans le second commit (**feat #2 - add cards design to the form options**), l'objectif était d'abandonner les simples boutons radios natifs qui formaient la structure du MVP, au profit de blocs cliquables esthétiques ("cards") riches en données. Pour accomplir cela sans dénaturer la sécurité du Form API de Drupal, le **BookingService.php** a été modifié afin de produire des variables **Markup** HTML encapsulées pour chaque option.

2.1 Injection de la Librairie CSS (**booking.libraries.yml** & **booking-cards.css**)

Nous avons créé une librairie Drupal pour y attacher la mise en page CSS Grid des Cards :

```
1 form_cards:
2   version: VERSION
3   css:
4     theme:
5       css/booking-cards.css: {}
```

```
1 /* css/booking-cards.css */
2 .booking-cards-wrapper .form-radios {
3   display: grid;
4   grid-template-columns: repeat(auto-fill, minmax(250px, 1fr));
5   gap: 1.5rem;
6 }
7 .booking-cards-wrapper .form-type-radio input[type="radio"] {
8   position: absolute; opacity: 0; width: 0; height: 0;
9 }
10 .booking-cards-wrapper .form-type-radio label {
11   display: flex; flex-direction: column; padding: 1.5rem;
12   border: 2px solid #e2e8f0; border-radius: 8px; cursor: pointer;
13 }
14 .booking-cards-wrapper .form-type-radio input[type="radio"]:checked + label {
15   border-color: #3182ce; background-color: #ebf8ff;
16 }
```

2.2 Injection HTML Sécurisée (**BookingService.php**)

```
1 public function getAgencyRichOptions(): array
2 {
3   $agencies = $this->entityTypeManager->getStorage('agency')->loadMultiple();
4   $options = [];
5   foreach ($agencies as $agency) {
6     $card = '<div class="booking-card" style="height: 100%;">';
7     $card .= '<h4 class="booking-card-title">' . htmlspecialchars((string) $agency->label()) . '</h4>';
8
9     if ($agency->hasField('address') && !$agency->get('address')->isEmpty()) {
10       $card .= '<div class="booking-card-detail"><strong>' . $this->t('Address') . ' :</strong> ' . htmlspecialchars((string)
11       $agency->get('address')->value) . '</div>';
12     }
13     $card .= '</div>';
14
15     // Drupal \Drupal\Core\Render\Markup permet le formatage de HTML dans les labels radio !
16     $options[$agency->id()] = \Drupal\Core\Render\Markup::create($card);
17   }
18   return $options;
19 }
```

2.3 Modification du Formulaire (**BookingFormTrait.php**)

Nous ajoutons simplement les identifiants CSS (**#attributes**) et la librairie (**#attached**) à chaque étape :

```
1 public function buildAgencyOptionsStep(array &$form, FormStateInterface $form_state)
2 {
3   $options = $this->bookingService->getAgencyRichOptions(); // Changement de fonction
4
5   $form['agency_options'] = [
6     '#type' => 'radios',
7     '#title' => $this->t('Select an agency'),
8     '#options' => $options,
9     '#attributes' => ['class' => ['booking-cards-wrapper']],
10    '#attached' => [
11      'library' => ['booking/form_cards'],
12    ],
13    // ...
14  ];
```

2.4 Démonstration Visuelle du Nouveau Design (Cards)

Select an agency *

Agadir Central Office
Address: Avenue Mohammed V, Agadir 80000
Phone: 0528123456
Hours: Mon–Fri 09:00–18:00

Talborjt Branch
Address: Quartier Talborjt, Agadir
Phone: 0528654321
Hours: Mon–Fri 08:30–16:30

Anza Tech Hub
Address: Zone Industrielle Anza, Agadir
Phone: 0528998877
Hours: Mon–Sat 09:00–12:00

FIGURE 11 – Design Interactif (Cards) : Agences avec métadonnées

Select an adviser *

jane_adviser
Email: jane_adviser@example.com

« Back

Next »

FIGURE 12 – Design (Cards) : Conseillers avec métadonnées email

Select a service *

Follow-up Meeting

General Consultation

Initial Interview

Technical Support

FIGURE 13 – Design (Cards) : Services proposés avec descriptions

CHAPITRE 3 : FILTRAGE DYNAMIQUE DES CONSEILLERS

Dans ce troisième cycle de développement (*Commit 3*), nous avons implémenté une fonctionnalité métier cruciale : s'assurer que les Conseillers (Advisers) affichés lors de l'étape 2 n'appartiennent exclusivement qu'à l'Agence sélectionnée par l'utilisateur lors de l'étape 1.

3.1 Modélisation de la Relation (User → Agency)

Puisque les conseillers sont de simples entités natives de type Utilisateur (**User**), nous devons d'abord créer un point d'attache structurel. Un script automatisé (implémenté ensuite via **booking.install**) a été exécuté pour provisionner informatiquement un champ **field_agency** sur le profil utilisateur. Ainsi, les gestionnaires peuvent facilement assigner des conseillers à des agences précises sous "People".

3.2 Filtrage Backend (**BookingService.php**)

La méthode de récupération des conseillers a été mise à jour en profondeur pour accepter un filtre optionnel **\$agencyId** et réduire la sélection :

```
1 public function getAdviserRichOptions($agencyId = NULL): array
2 {
3     $properties = ['status' => 1];
4     if ($agencyId) {
5         $properties['field_agency'] = $agencyId;
6     }
7
8     // Seuls les conseillers liés à l'agence spécifiée sont chargés.
9     $users = $this->entityTypeManager->getStorage('user')->loadByProperties($properties);
10
11     // ... Génération du design des Cards
12 }
```

3.3 Transmission de l'État Inter-Étapes (**BookingFormTrait.php**)

Puisque le formulaire Ajax conserve les valeurs saisies de manière asynchrone à chaque étape via le **FormState**, le trait du formulaire s'occupe de lire la valeur archivée lors de l'étape 1 (**agency_options**) et l'injecte dans le contrôleur :

```
1 public function buildAdviserStep(array &$form, FormStateInterface $form_state)
2 {
3     $stored = $form_state->get('stored_values') ?: [];
4     $agencyId = $stored['agency_options'] ?? NULL;
5
6     $options = $this->bookingService->getAdviserRichOptions($agencyId);
7
8     // Vérification d'erreur UX
9     if (empty($options)) {
10         $this->messenger()->addWarning($this->t('No advisers available for the selected agency.'));
11     }
12
13     // ... Rendu du champ
14 }
```

3.4 Démonstration Visuelle du Filtrage

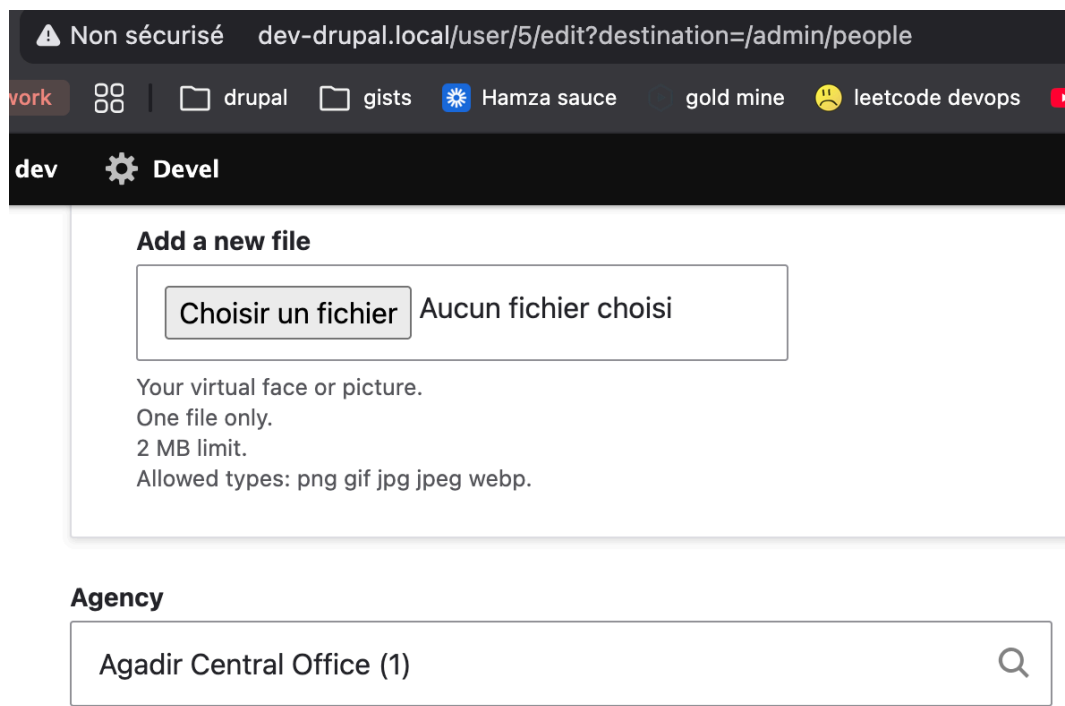


FIGURE 14 – Assignment d'une agence a un compte Utilisateur dans l'espace Administration

CHAPITRE 4 : FILTRES, TRIS ET PAGINATION ADMINISTRATEUR

Dans ce quatrième commit (*feat #4 - add admin filter, sort, search, pagination*), l'objectif était d'améliorer considérablement la vue d'administration (**BookingListBuilder**) en y ajoutant des capacités de recherche complexes, de tris par colonnes, et de pagination automatique.

4.1 Architecture : Constructeur et Rendu

Afin d'implémenter ces fonctionnalités, l'architecture du **ListBuilder** a dû être adaptée via l'injection de dépendances et la surcharge de méthodes clés.

Pourquoi injecter **EntityTypeInterface** et **EntityStorageInterface** ? Nativement, la classe parente **EntityListBuilder** utilise un constructeur précis. Pour modifier la propriété interne limitant le nombre d'éléments par page (`$this->limit` qui est par défaut de 50), il nous fallait déclarer notre propre constructeur `__construct`, appeler `parent::__construct()` pour satisfaire l'injection de dépendance de Drupal, puis redéfinir la limite à 10 :

```
1 public function __construct(EntityTypeInterface $entity_type, EntityStorageInterface $storage) {
2     parent::__construct($entity_type, $storage); // Satisfait le Core de Drupal
3     $this->limit = 10; // Ecrase la limite de la pagination
4 }
```

Pourquoi surcharger la méthode `render()` ? Avant ce commit, **BookingListBuilder** affichait uniquement un tableau. La méthode `parent::render()` s'en chargeait parfaitement. Or, nous voulions physiquement injecter un nouveau formulaire de filtrage (**BookingFilterForm**) au-dessus du tableau. En surchargeant `render()`, nous construisons d'abord le formulaire, puis nous appelons le parent pour y rattacher la table de résultats en dessous :

```
1 public function render() {
2     $build['filter_form'] = \Drupal::formBuilder()->getForm('\Drupal\booking\Form\BookingFilterForm');
3     $build += parent::render();
4     return $build;
5 }
```

4.2 Logique de la Pagination, Tris et Recherche

Tout le secret de ces trois fonctionnalités réside dans la méthode `getEntityIds()`. C'est ici que la requête SQL à la base de données est interceptée et dynamisée.

1. La Pagination

Au lieu d'écrire une logique complexe de décalage (offset) manuel, le moteur de requêtes de Drupal accepte de simples déclarations. En chaînant `->pager($this->limit)`, Drupal est instruit de charger uniquement 10 réservations à la fois (soit un SQL `LIMIT 10 OFFSET X`). En prime, le bloc HTML natif de pagination (`[1] [2] [3] Suivant`) est généré automatiquement en pied de page !

2. Le Tri par Colonnes (Sorting)

Pour rendre les tableaux triables, la méthode `buildHeader()` a été redéfinie. Une simple chaîne `'Date'` a été transformée en un tableau `['data' => 'Date', 'field' => 'booking_date', 'sort' => 'desc']` pour créer un lien `<a>` cliquable qui passe `?sort=asc` dans l'URL du navigateur. Ensuite, dans `getEntityIds()`, on injecte les entêtes :

```
1 $header = $this->buildHeader();
2 $query->tableSort($header); // Drupal lit l'URL et genere `ORDER BY`
```


3. Le Filtrage et la Recherche (Search)

Quand l'administrateur valide le **BookingFilterForm**, la page se recharge avec des requêtes GET (ex : `?search=Marie&status=pending`).

Dans `getEntityIds()`, on lit ces paramètres HTTP et on y accroche des conditions **WHERE** :

```
1 $request = \Drupal::request(); // Lecture de l'URL
2
3 // Recherche Textuelle Mutli-colonnes (SQL: LIKE)
4 if ($search = $request->query->get('search')) {
5     $orGroup = $query->orConditionGroup()
6     ->condition('reference', '%' . $search . '%', 'LIKE')
7     ->condition('booking_customer_name', '%' . $search . '%', 'LIKE');
8     $query->condition($orGroup); // Trouve Reference OU Client
9 }
10
11 // Filtrage Stricte par statut (SQL: = 'status')
12 if ($status = $request->query->get('status')) {
13     $query->condition('booking_status', $status);
14 }
```

4.3 Démonstration Visuelle de l'Administration

The screenshot displays the 'BookingFilterForm' interface. At the top, there are three input fields: 'Search' (containing 'Ref or Customer...'), 'Status' (a dropdown menu showing '- All -'), and 'Agency' (a dropdown menu showing '- All -'). Below these fields is a grey 'Filter' button. Underneath the button is a table header with the following columns: 'Reference' (with an upward arrow icon), 'Customer' (with a horizontal line icon), 'Date' (with a horizontal line icon), 'Status' (with a horizontal line icon), and 'Agency' (with a horizontal line icon). The 'Reference' column is currently selected and highlighted with a blue underline.

FIGURE 15 – Interface Complète : Formulaire de filtrage, Recherche et Entêtes triables



FIGURE 16 – Le module de Pagination natif de Drupal, injecté en pied de page

CHAPITRE 5 : REFACTORISATION ET SERVICE D'EXPORT CSV

Dans ce cinquième commit (*feat #5 - refactor service namespaces and implement csv export*), nous avons entrepris une restructuration majeure de l'architecture du module pour isoler les responsabilités et permettre l'exportation des données.

5.1 Conformité PSR-4 et Namespaces

L'un des changements critiques a été le déplacement du **BookingService.php** du dossier racine **src/** vers un sous-dossier **src/Service/**. En Drupal, ce déplacement physique impose une mise à jour du **namespace** pour respecter la norme PSR-4 utilisée par l'autoloader.

- Ancien Namespace : `namespace Drupal\booking;`
- Nouveau Namespace : `namespace Drupal\booking\Service;`

L'enregistrement dans le fichier **booking.services.yml** a également dû être mis à jour pour pointer vers le nouveau chemin de classe :

```
1 services:
2   booking.service:
3     class: Drupal\booking\Service\BookingService
4     # ... arguments
```

Sans cette correction, Drupal ne pourrait plus localiser la classe, provoquant des erreurs fatales sur tout le site. Toutes les classes dépendantes (Formulaires, Validateurs) ont également vu leurs instructions **use** mises à jour.

5.2 Architecture MVC du module d'Export

Pour l'exportation CSV, nous avons adopté un découplage strict entre la logique métier et la réponse HTTP. Le **BookingController** agit comme chef d'orchestre :

```
1 class BookingController extends ControllerBase {
2   // ... injection du ExportService
3   public function exportData() {
4     $response = new StreamedResponse(function () {
5       $this->exportService->prepareDataForExport();
6     });
7     $response->headers->set('Content-Type', 'text/csv; charset=utf-8');
8     return $response;
9   }
10 }
```

Le **ExportService** gère quant à lui l'ouverture du flux PHP (`php://output`) et la conversion des entités en lignes CSV.

CHAPITRE 6 : OPTIMISATION MÉMOIRE ET BATCHING

Le sixième commit (*fix #6 - implement memory-efficient csv export with batches*) est venu corriger un problème potentiel de performance : la montée en charge de la mémoire lors de l'exportation de milliers de réservations.

6.1 Le Problème du static cache de Drupal

Par défaut, chaque entité chargée par Drupal reste en mémoire vive (cache statique) jusqu'à la fin de l'exécution du script. Pour un export massif, cela peut saturer la RAM du serveur.

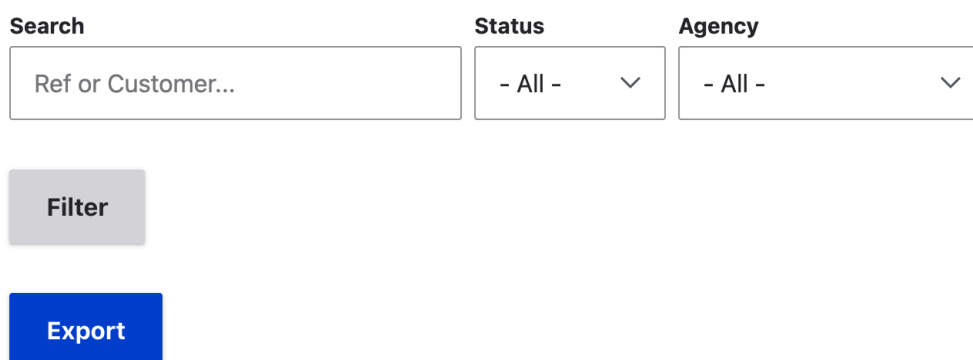
6.2 Solution : Chargement par Chunks et Reset Cache

Nous avons implémenté une boucle de traitement par "paquets" (chunks) de 50 entités. Le secret de l'économie de mémoire réside dans l'instruction **resetCache()** :

```
1 $chunkSize = 50;
2 $offset = 0;
3 while (TRUE) {
4     $ids = $storage->getQuery()
5         ->range($offset, $chunkSize)
6         ->execute();
7
8     if (empty($ids)) break;
9
10    $bookings = $storage->loadMultiple($ids);
11
12    foreach ($bookings as $booking) {
13        fputcsv($handle, [ ... ]); // Ecriture immediate
14    }
15
16    $offset += $chunkSize;
17    $storage->resetCache($ids); // Libere la memoire vive !
18 }
```

Grace à **StreamedResponse** et **resetCache()**, le module est désormais capable d'exporter des millions de lignes avec une consommation de mémoire constante et minimale.

6.3 Démonstration de l'Export



The screenshot shows a web interface for generating a CSV file. It features three filter sections: 'Search' with a text input 'Ref or Customer...', 'Status' with a dropdown menu showing '- All -', and 'Agency' with a dropdown menu showing '- All -'. Below these filters are two buttons: a grey 'Filter' button and a blue 'Export' button.

FIGURE 17 – Visualisation du fichier CSV genere via l'interface d'administration

CHAPITRE 7 : SYMFONY MAILER ET NOTIFICATIONS DE STATUT

Le septième commit (*feat #7 - Implement email verification and status notifications*) marque la mise en place d'un système de communication automatisé. Chaque changement d'état du rendez-vous déclenche l'envoi d'un email personnalisé via l'API Symfony Mailer.

7.1 Interception des Changements (`hook_entity_presave`)

La détection du changement de statut se fait en comparant l'état actuel avec l'état précédent stocké dans `$entity->original`.

```
1 /**  
2  * Implements hook_entity_presave().  
3  */  
4 function booking_booking_presave(EntityInterface $entity) {  
5     if ($entity->getEntityTypeId() !== 'booking') return;  
6  
7     if (isset($entity->original)) {  
8         $old_status = $entity->original->get('booking_status')->value;  
9         $new_status = $entity->get('booking_status')->value;  
10  
11         if ($old_status !== $new_status) {  
12             // Appel au service de notification  
13             \Drupal::service('booking.mailer')->sendStatusUpdate($entity);  
14         }  
15     }  
16 }
```

7.2 Vérification et Notifications Visuelles

From: Drupal Dev Environment <admin@example.com>
Subject: **Mise à jour de votre réservation : REF-20260321-C85A**
To: y.elomari@void.fr

HTML Plain text Source MIME

Bonjour, Le statut de votre réservation a été mis à jour. Nouveau statut : Annulé. Merci de votre confiance.

FIGURE 18 – Email reçu automatiquement après le passage d'une réservation au statut 'Confir-mé'.

From: Drupal Dev Environment <admin@example.com>
Subject: **Votre code de vérification pour la réservation REF-20260321-C85A**
To: y.elomari@void.fr

HTML Plain text Source MIME

Bonjour, Vous avez demandé à modifier votre réservation. Voici votre code de vérification unique :

141315

Ce code est strictement personnel. Si vous n'êtes pas à l'origine de cette demande, merci d'ignorer cet e-mail.

FIGURE 19 – Code de vérification envoyé au client pour accéder à la modification de sa réservation.

CHAPITRE 8 : VALIDATION DES TRANSITIONS ET ANNULATION

Le huitième commit (*fix #8 - enhance booking status transition validation*) a introduit une couche de sécurité supplémentaire via des Validation Constraints. L'objectif est d'empêcher les transitions de statut incohérentes (ex : passer de 'Annulé' à 'Confirmé').

8.1 Valideur de Transition (**ValidBookingStatusTransitionConstraintValidator.php**)

```
1 public function validate($item, Constraint $constraint) {
2     $entity = $item->getEntity();
3     if (!isset($entity->original)) return;
4
5     $old_status = $entity->original->get('booking_status')->value;
6     $new_status = $item->value;
7
8     // Regles métier : on ne peut pas modifier un rendez-vous déjà annulé ou terminé
9     if (in_array($old_status, ['cancelled', 'completed', 'deleted'])) {
10         $this->context->addViolation($constraint->message);
11     }
12 }
```

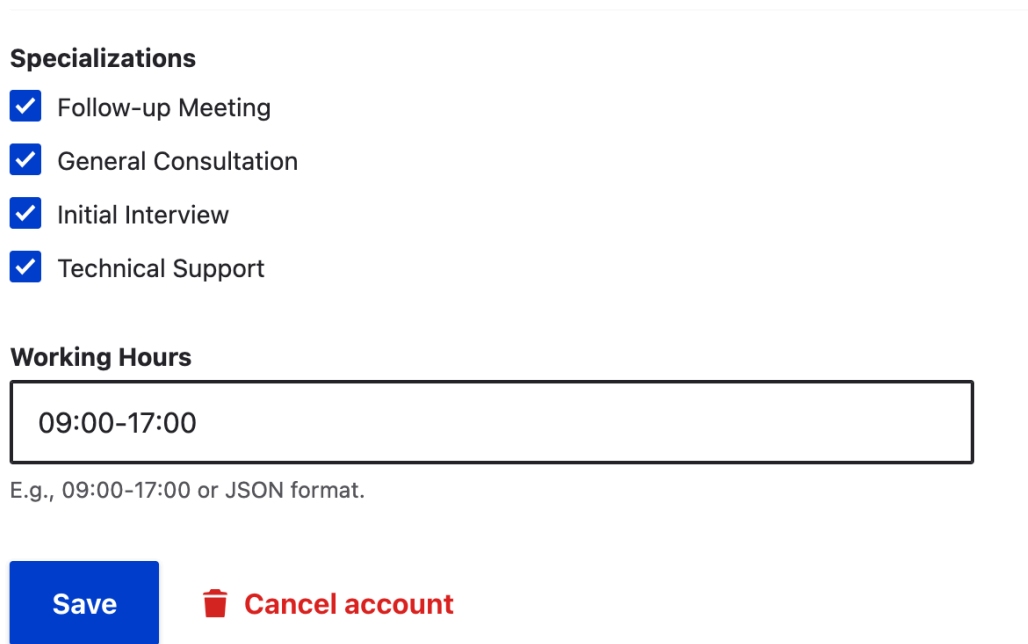
CHAPITRE 9 : PROFILING CONSEILLER ET SPÉCIALISATIONS

Dans le neuvième commit (**feat #9 - add deleted status, user fields and specialization filtering**), nous avons transformé les comptes utilisateurs en profils de conseillers spécialisés.

9.1 Extension du Profil Utilisateur

Nous avons utilisé le système de champs configurables de Drupal pour ajouter :

- Spécialités : Un champ de référence vers les termes de taxonomie.
- Horaires de travail : Pour définir les disponibilités de chaque conseiller.



The screenshot shows a user profile form with two main sections: 'Specializations' and 'Working Hours'. The 'Specializations' section has four checkboxes, all of which are checked: 'Follow-up Meeting', 'General Consultation', 'Initial Interview', and 'Technical Support'. The 'Working Hours' section has a text input field containing '09:00-17:00'. Below the input field is a small text hint: 'E.g., 09:00-17:00 or JSON format.' At the bottom of the form are two buttons: a blue 'Save' button and a red 'Cancel account' button with a trash icon.

Specializations

- ☒ Follow-up Meeting
- ☒ General Consultation
- ☒ Initial Interview
- ☒ Technical Support

Working Hours

09:00-17:00

E.g., 09:00-17:00 or JSON format.

Save  **Cancel account**

FIGURE 20 – Nouveaux champs de spécialisation ajoutés au profil des utilisateurs.

CHAPITRE 10 : GESTION DES CRÉNEAUX HORAIRES EN 1 HEURE

Le dixième commit (*feat #10 - add 1-hour time-slot*) a radicalement changé la planification. Au lieu d'une saisie libre de l'heure, le système génère désormais des créneaux fixes d'une heure.

10.1 Logique de Découpage (**BookingService.php**)

Le service calcule les créneaux disponibles en vérifiant chaque heure par rapport aux réservations existantes. Si une réservation (de n'importe quelle durée) existe dans une tranche horaire, le créneau complet est bloqué.

```
1 // On split le temps de travail par tranches de 1h
2 for ($t = $start; $t < $end; $t += 3600) {
3     $slot = date('H:i', $t);
4     if (!in_array($slot, $busySlots)) {
5         $available[] = $slot;
6     }
7 }
```

10.2 Rendu UI des Créneaux

Time slot (1 hour) *

09:00 - 10:00

11:00 - 12:00

12:00 - 13:00

13:00 - 14:00

14:00 - 15:00

FIGURE 21 – Affichage des créneaux de 1h sous forme de sélecteur visuel pour le client.

Customer email *

y.elomari@void.fr

Phone number *

0708521209

Appointment Date

13/03/2026 

Available Time Slots (1 hour) *

☒ 09:00 - 10:00

☐ 11:00 - 12:00

☐ 12:00 - 13:00

☐ 13:00 - 14:00

☐ 14:00 - 15:00

☐ 15:00 - 16:00

☐ 16:00 - 17:00

FIGURE 22 – Interface d'édition administrative utilisant le sélecteur de créneaux intelligent.

CHAPITRE 11 : INTERFACE INTERACTIVE AVEC FULLCALENDAR

Pour offrir une expérience de classe mondiale, le onzième commit (*feat #11 - integrate FullCalendar*) a remplacé le sélecteur de date natif par une interface de calendrier interactive.

11.1 Pont JavaScript et AJAX (**booking-calendar.js**)

Le défi était de lier les clics sur le calendrier (bibliothèque externe) au système AJAX de Drupal. Nous avons utilisé un champ **textfield** masqué et déclenché un événement **change** manuel.

```
1 select: function(info) {  
2   const dateStr = info.startStr;  
3   $('input[name="booking_date_selection"]').val(dateStr).trigger('change');  
4 }
```

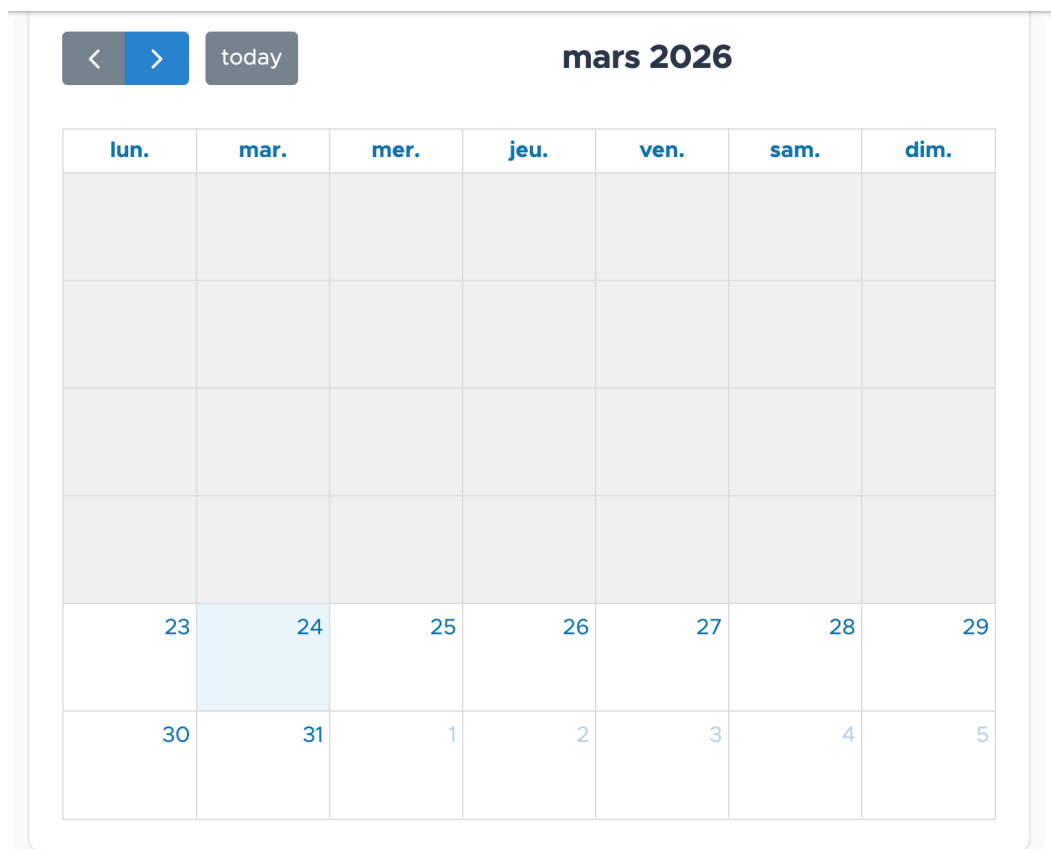


FIGURE 23 – Intégration de FullCalendar pour une sélection de date fluide et visuelle.

CHAPITRE 12 : SYSTÈME DE SOFT DELETE ET SÉCURITÉ

Le dernier commit (*feat #12 - implement soft delete logic*) a introduit la suppression non-destructive ("Soft Delete") pour maintenir l'intégrité des données tout en libérant les créneaux.

12.1 Suppression Logique et Access Control

Au lieu de supprimer l'entité, nous changeons son statut vers **DELETED**. De plus, nous avons ajouté un gardien dans le formulaire d'accès client (**BookingEditForm**) pour bloquer toute consultation d'un rendez-vous supprimé.

```
1 // Check si le rendez-vous est supprimé
2 if ($booking->get('booking_status')->value === 'deleted') {
3     $this->messenger()->addError(t('Deleted reservation cannot be accessed.'));
4     return;
5 }
```

12.2 Visualisation du Soft Delete

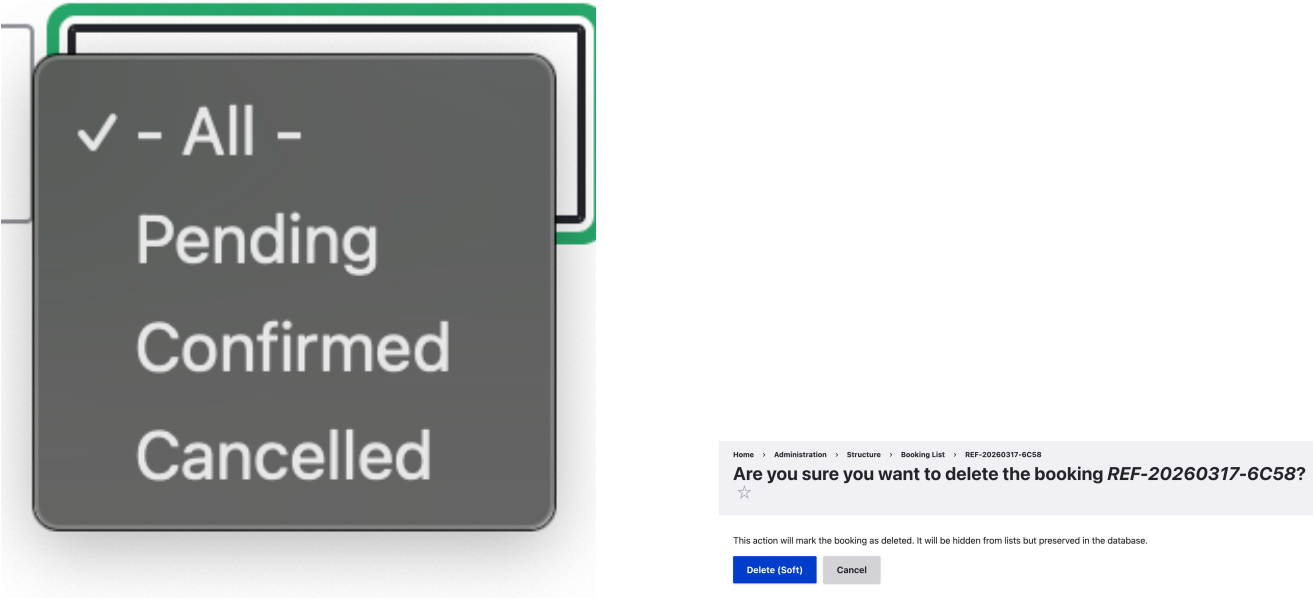


FIGURE 24 – Statut 'Supprimé' en base de données et processus de confirmation administratif.

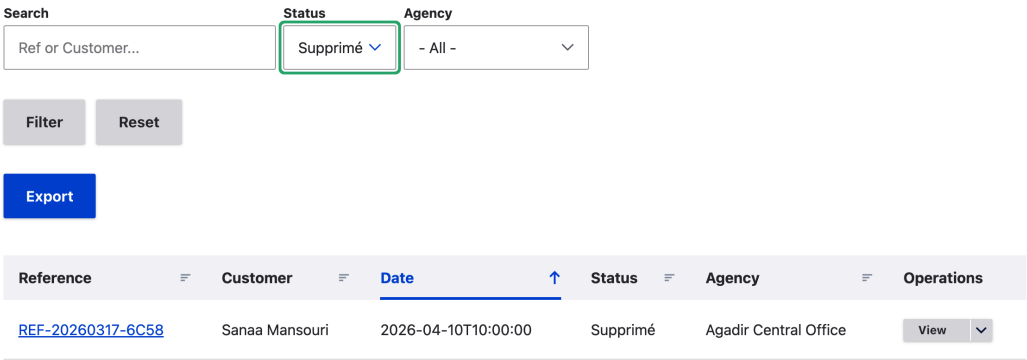


FIGURE 25 – Blocage de sécurité : un client ne peut plus accéder à un rendez-vous supprimé.

CHAPITRE 13 : SYSTÈME DE PERMISSIONS ET RÔLES

Le commit (*feat #12 - add roles and permissions*) a finalisé la sécurité du module en introduisant des permissions spécifiques et un contrôleur d'accès au niveau de l'entité.

13.1 Définition des Permissions (`booking.permissions.yml`)

Nous avons défini des droits granulaires pour séparer les actions administratives des actions publiques :

- Administration : `access booking overview`, `add booking entity`, `edit booking entity`, `delete booking entity`, `export booking data`.
- Public :
 - `create booking` : Permet aux clients d'accéder au formulaire de réservation (`/prendre-un-rdv`).
 - `edit own booking` : Permet aux clients de gérer leur propre réservation (`/mes-rdv`) via le système de code secret.

13.2 Contrôleur d'Accès Entité (`BookingAccessControlHandler.php`)

Le contrôleur d'accès a été configuré pour autoriser les opérations `create` et `update` si l'utilisateur possède les permissions publiques correspondantes. Cela garantit une protection robuste, même en dehors des formulaires standards.

13.3 Configuration des Rôles Recommandés

Voici la structure de rôles mise en place pour une exploitation sécurisée :

- Assistant (Lecture seule) : `access booking overview`.
- Manager (Opérations) : Toutes les permissions d'administration sauf la configuration globale.
- Admin (Full) : `administer booking`.

Permission	Anonymous user	Authenticated user	Content editor	Administrator	ex-port-Role	Booking Manager	Booking Admin
Access booking overview View the administrative list of all bookings .	☐	☐	☐	☒	☐	☒	☒
Add new booking Create a new booking via the administrative interface.	☐	☐	☐	☒	☐	☒	☒
Administer Booking configuration <i>Warning: Give to trusted roles only; this permission has security implications.</i> Manage all settings and configurations for the booking module.	☐	☐	☐	☒	☐	☐	☒
Create booking Use the public booking form to schedule a rendezvous.	☒	☒	☒	☒	☒	☒	☒
Delete booking Soft-delete bookings (change status to deleted).	☐	☐	☐	☒	☐	☒	☒
Edit booking Modify details of existing	☐	☐	☐	☒	☐	☒	☒

FIGURE 26 – Interface de gestion des permissions Drupal pour le module Booking.

13.4 Sécurisation des Routes (**booking.routing.yml**)

Toutes les routes d'administration (**/admin/structure/booking/...**) ont été mises à jour pour exiger ces nouvelles permissions.